

一、同步机制 (Synchronization)

- **信号量计算 (Semaphore Calculation):**
 - **最终值 = 初始值 + signal()次数 - wait()次数**。若最终值 < 0 ，则有进程被阻塞。(注意分辨代码循环)。[24Fall-MC1, 21Spring-MC1]
- **自旋锁 vs 互斥锁 (Spinlock vs Mutex):**
 - **自旋锁 (Spinlock)** 不会引发上下文切换睡眠，适合**短时间持有**。其效率上限 (Upper bound) 通常为 $2T$ (T 为一次上下文切换时间)。不能在持有自旋锁时请求信号量 (会导致睡眠/死锁)。[24Spring-MC1.1, MC1.2]
- **自适应互斥锁 (Adaptive Mutex):**
 - 若持有锁的进程正在其他CPU运行 $\rightarrow$ **自旋 (Spin)**；若持有锁的进程未运行 $\rightarrow$ **睡眠 (Sleep)**。[23Fall-MC1.1, 24Spring-MC1.3]
- **条件变量 (Condition Variables):**
 - 必须在 **while** 循环中使用 **wait** (防止虚假唤醒 Spurious wakeup)，不能用 **if**。[25Fall-MC18]
- **读者-写者问题 (Readers-Writers):**
 - **rw_mutex**：写者和**第一个**读者竞争。**mutex**：保护 **read_count**。
 - 无写者申请时，读者不应等待；**可能导致写者饥饿 (Starvation)**。[21Spring-MC2, 23Fall-MC1.2, 24Fall-MC3, 25Fall-MC19]

二、死锁 (Deadlock)

- **状态定义 (State Definitions):**
 - **安全状态 (Safe)** $\neq$ **死锁**。系统在安全状态**绝对不会**死锁。
 - **不安全状态 (Unsafe)** $\neq$ **死锁**。不安全状态**可能 (may)** 导致死锁，但不代表当前已死锁。[21Spring-MC4, 23Spring-MC1.1, 24Fall-MC5, 25Fall-MC1]
- **资源分配图 (RAG) 与环 (Cycle):**
 - 每类资源**仅有1个实例**：有环 $\leftrightarrow$ 死锁 (充分必要)。
 - 每类资源**有多个实例**：有环 $\rightarrow$ **可能死锁** (非充分，仅为必要条件)。[21Spring-MC4, 23Spring-MC1.2]
- **死锁预防 (Deadlock Prevention) 破坏四大条件:**
 - **Hold and Wait (占有并等待)**：要求进程一次性申请所有资源，或申请新资源前释放所有资源。[24Fall-MC4, 25Spring-MC1]
 - **Circular Wait (循环等待)**：对所有资源类型**强制全局排序 (Ordering)**，按递增顺序申请。[21Spring-MC3, 25Spring-MC1, 25Fall-MC3]
- **银行家算法 (Banker's Algorithm) 计算:**
 - **Need = Max - Allocation**
 - 第一步检查：**Request $\leq$ Need** 且 **Request $\leq$ Available**。[25Spring-MC2]
- **死锁避免/安全序列极简推算:**
 - **最少需要总资源数 = 当前已分配总量 (Sum of Alloc) + 能让需求最小的进程运行完毕的 Need**。[23Spring-MC1.3, 24Spring-MC1.5, 25Fall-MC2]
- **死锁检测 (Deadlock Detection):**
 - 复杂度 $O(m \times n)$ 。若初始 **Allocation == 0**，则直接设 **Finish[i] = true**。[24Fall-MC6, 25Spring-MC4]

三、内存管理 (Memory Management)

- **绑定时机 (Binding):** 大多数通用OS采用 **执行时绑定 (Execution time binding)**。[21Spring-MC8, 23Spring-MC1.5]
- **碎片 (Fragmentation):**
 - **外部碎片 (External):** 空闲空间不连续 (**Best-fit** 最容易产生极小外部碎片)。
 - **内部碎片 (Internal):** 分页系统/大页面带来, 分配的空间大于实际需求。[21Spring-MC6, 23Spring-MC1.6, 23Fall-MC1.11]
- **基址与限长寄存器 (Base & Limit Registers):**
 - 合法地址条件: $0 \leq \text{Logical Address} < \text{Limit}$ 。越界会导致 Trap。物理地址 = $\text{Base} + \text{Logical}$ 。[25Spring-MC5]
- **地址转换拆分规律 (Paging Address Structure):**
 - 页偏移(Offset) = $\log_2(\text{Page Size})$
 - 页号(Page Number) = 逻辑地址总位数 - Offset
 - 每一级页表的索引位数 $p_i = \log_2(\text{一页能存几个PTE})$ 。[23Spring-MC1.8, 24Fall-MC9, MC10, 25Spring-MC7, 25Fall-MC4, MC5]
- **多级页表的目的: 避免页表在内存中连续分配** (Paging the page table), 而非减少页表条目总数。[23Spring-MC1.7, 25Spring-MC9]

四、虚拟内存 (Virtual Memory)

- **有效访问时间 (Effective Access Time, EAT) 计算:**
 - **纯TLB场景:** $EAT = \alpha \times (T_{tlb} + T_{mem}) + (1 - \alpha) \times (T_{tlb} + k \times T_{mem})$ (其中 k 为访问页表的次数, 单级为2, 多级叠加)。[21Spring-MC7, 23Fall-MC1.8, 24Fall-MC8]
 - **带缺页的场景:** $EAT = (1 - P_{fault}) \times T_{mem_access} + P_{fault} \times T_{page_fault}$ 。[24Fall-MC11, 25Fall-MC9]
- **Belady's Anomaly (Belady异常):**
 - **特征:** 分配更多帧 (Frames), 反而引发更多缺页。
 - **适用性:** 只有 **FIFO** 会出现。栈算法 (**LRU, OPT**) 绝对免疫。[21Spring-MC9, 23Spring-MC1.14, 24Fall-MC13, 25Spring-MC12]
- **系统抖动 (Thrashing):**
 - 表现为页面频繁换入换出, 随着多道程序度 (multiprogramming) 增加, CPU 利用率骤降。[21Spring-MC11, 23Spring-MC1.12, 24Fall-MC14]
- **工作集模型 (Working-Set Model):**
 - 窗口 Δ 太小 $\rightarrow$ 无法覆盖局部性, 导致 thrashing。
 - 窗口 Δ 太大 $\rightarrow$ 包含了多个不同的局部性 (distinct localities) 甚至整个程序。[25Spring-MC11]
 - **TLB Reach** = TLB条目数 $\times$ 页面大小。必须 $\geq$ 当前工作集。[24Spring-MC1.11]

五、磁盘与存储 (Mass Storage)

- **平均 I/O 时间与带宽 (I/O Time & Bandwidth):**
 - 平均旋转延迟(Avg Latency) = $0.5 \times \frac{60}{\text{RPM}}$ 秒。
 - 单次I/O耗时 = 寻道时间(Seek) + 平均延迟 + $\frac{\text{数据量}}{\text{传输速率}}$ + 控制器开销。
 - 有效带宽 = $\frac{\text{数据量}}{\text{总耗时}}$ 。[21Spring-MC12, 23Fall-MC1.14, 24Fall-MC15, 25Spring-MC13, 25Fall-MC15]
- **磁盘调度算法 (Disk Scheduling):**
 - **SCAN** (电梯算法): 往一个方向走到磁盘端点 (如 0 或 max) 再折返。
 - **C-SCAN**: 走到端点后, 直接飞回另一端尽头, 返回途中不服务请求。

- **LOOK / C-LOOK**: 走到最远请求处即折返, 不碰端点。[23Fall-MC1.15, 24Fall-MC16, 24Spring-MC1.16, 25Spring-MC14, 25Fall-MC13]

六、文件系统 (File System)

- **文件分配表 (FAT, File Allocation Table):**
 - 相较于传统链式分配 (Linked Allocation), FAT 可以被缓存在内存中, **支持高效的随机访问 (Random Access)**。
 - 文件大小推导: $\text{文件大小} = \text{占用FAT条目数} \times (\text{总磁盘容量} / 2^{\text{FAT位数}})$ 。[24Spring-MC1.19]
- **磁盘块计算 (Blocks Calculation):**
 - $\text{数据块数} = \text{ceil}(\text{文件大小} / \text{块大小})$
 - **连续分配 (Contiguous)**: 总块数 = 数据块数。
 - **索引分配 (Indexed)**: 总块数 = 数据块数 + 索引块数 ($\text{ceil}(\text{数据块数} / \text{每个块存放的指针数})$)。[23Fall-MC1.18, 25Fall-MC14]
- **打开文件表 (Open-File Tables):**
 - **进程级表 (Per-process)**: 进程打开的每一个文件占 1 项 (独立计算)。
 - **系统级表 (System-wide)**: 所有进程共享去重, 仅包含唯一的 FCB。维护 **file-open count**, 当计数为0时才移除。[23Fall-MC1.17, 24Fall-MC17, 24Spring-MC1.18, 25Spring-MC17, 25Fall-MC12]
- **目录结构 (Directory Structure):**
 - 无环图目录 (Acyclic-graph): 允许多个父目录共享同一个文件或子目录。[23Spring-MC1.17]

七、保护与安全 (Protection)

- **访问矩阵 (Access Matrix):**
 - **Owner**: 可以更改行/列中的权限 (通常控制对象列)。
 - **Copy (*)**: 只能在同一列内传递权限。
 - **Control**: 允许一个域更改另一个域 (行) 的访问权限。
 - **Switch**: 允许从一个域切换到另一个域。[23Fall-MC1.19, 24Fall-MC19, 25Spring-MC20]
- **访问控制列表 (Access List/ACL) vs 能力列表 (Capability List):**
 - **能力列表 (Capability List)**: 与域/进程关联 (Object + Rights 的凭证)。优点: 能防止混淆代理问题 (Confused Deputy Problem), 易于本地化。缺点: **难以撤销 (Revocation)** 和审计某个对象谁能访问。[21Spring-MC16, 23Fall-MC1.20, 24Fall-MC20, 25Fall-MC16, MC17]
 - **访问列表 (Access List / ACL)**: 与对象 (Object) 关联。优点: 极易撤销权限和审计。缺点: 查找某个进程拥有哪些权限很困难。[24Spring-MC1.20, 25Fall-MC16]